

CodeBLEU Similarity Analysis Report

1 Introduction

Code similarity measurement is crucial for evaluating code quality, plagiarism detection, and assessing LLM-generated code. CodeBLEU is an automated evaluation metric specifically designed for code synthesis tasks. Unlike traditional BLEU which was designed for natural language, CodeBLEU incorporates code-specific features including abstract syntax tree (AST) matching and data flow analysis to better capture code structural and semantic similarities. This report analyzes CodeBLEU similarity scores between correct code and two types of comparison code: LLM-generated code and faulty code, across five programming problems. The objective is to evaluate whether CodeBLEU accurately reflects semantic consistency between code implementations.

2 Methodology

2.1 CodeBLEU Metric

CodeBLEU extends traditional BLEU by incorporating four components [1]:

- **Weighted n-gram matching:** Enhances importance of keywords and reserved words
- **Syntax AST matching:** Compares abstract syntax tree structures
- **Data flow matching:** Analyzes variable definition-use relationships
- **Traditional BLEU:** Basic n-gram surface similarity

The final score is computed as: $\text{CodeBLEU} = \alpha \cdot \text{BLEU} + \beta \cdot \text{BLEU_weighted} + \gamma \cdot \text{Match_ast} + \delta \cdot \text{Match_df}$ with typical weights set to 0.25 for each component.

2.2 Experimental Setup

We computed CodeBLEU similarity for five programming problems:

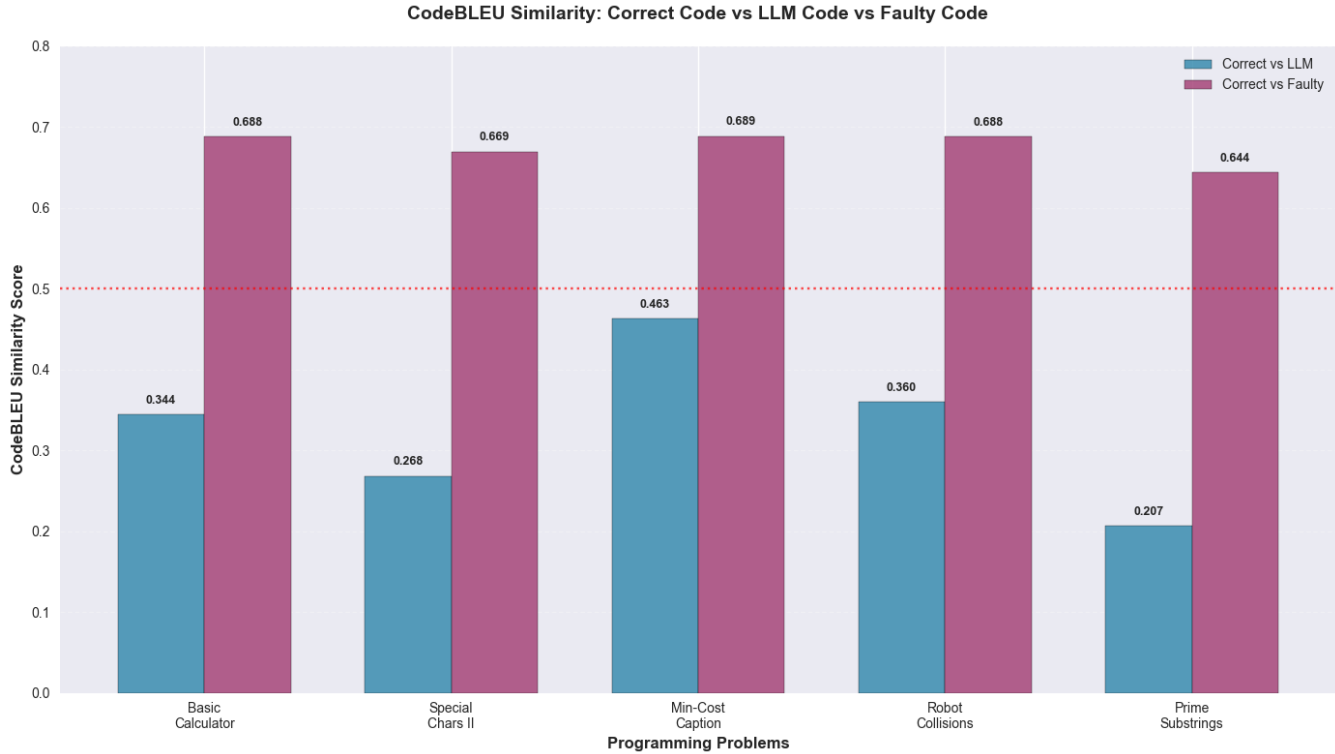
- **Comparison Pair 1:** Correct code vs. LLM-generated code
- **Comparison Pair 2:** Correct code vs. faulty code
- **Dataset:** Five problems of varying complexity including basic calculator, string processing, and algorithm implementation

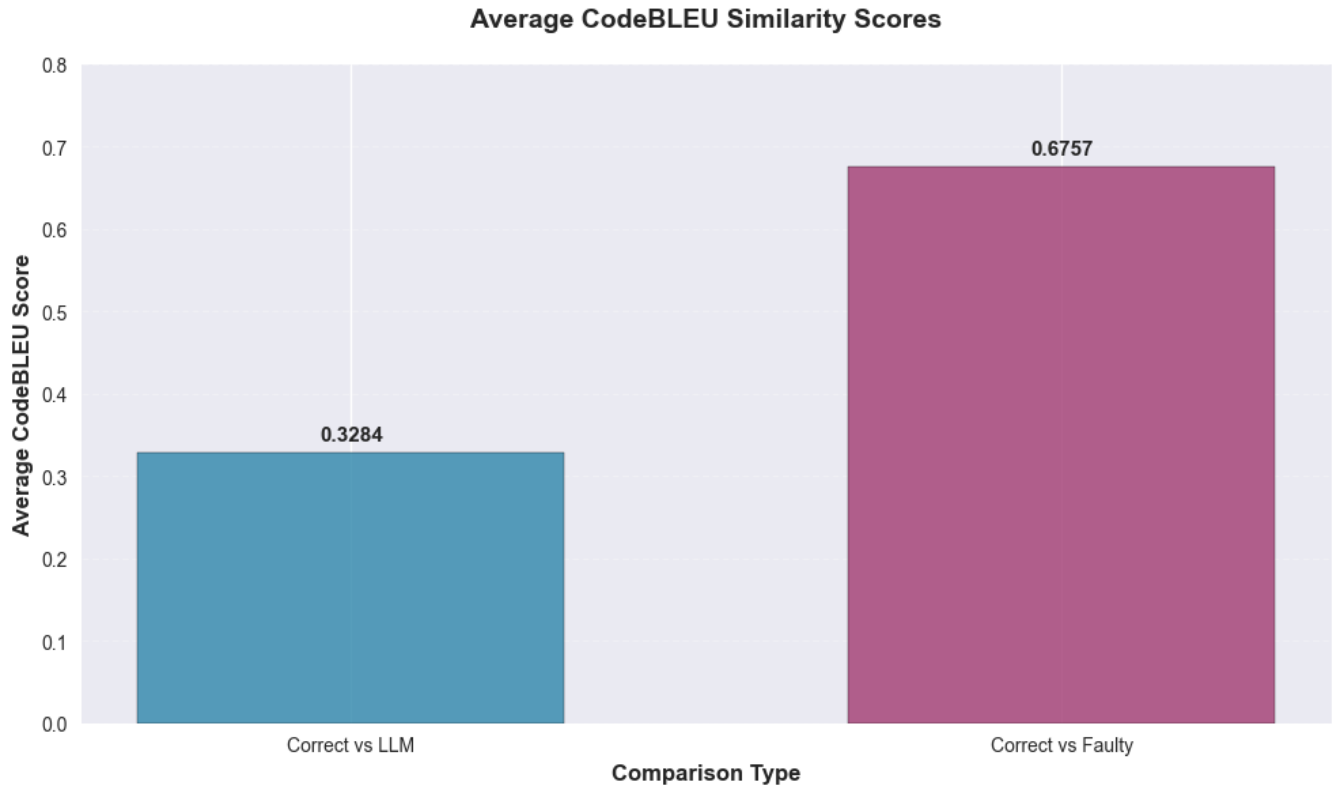
3 Results and Analysis

3.1 CodeBLEU Similarity Scores

The calculated CodeBLEU scores reveal a consistent pattern across all five problems:

Problem	Correct vs LLM	Correct vs Faulty	Score Difference
basic-calculator	0.3443	0.6883	-0.3440
count-the-number-of-special-characters-ii	0.2681	0.6695	-0.4014
minimum-cost-good-caption	0.4630	0.6889	-0.2259
robot-collisions	0.3601	0.6880	-0.3279
sum-of-largest-prime-substrings	0.2066	0.6440	-0.4374



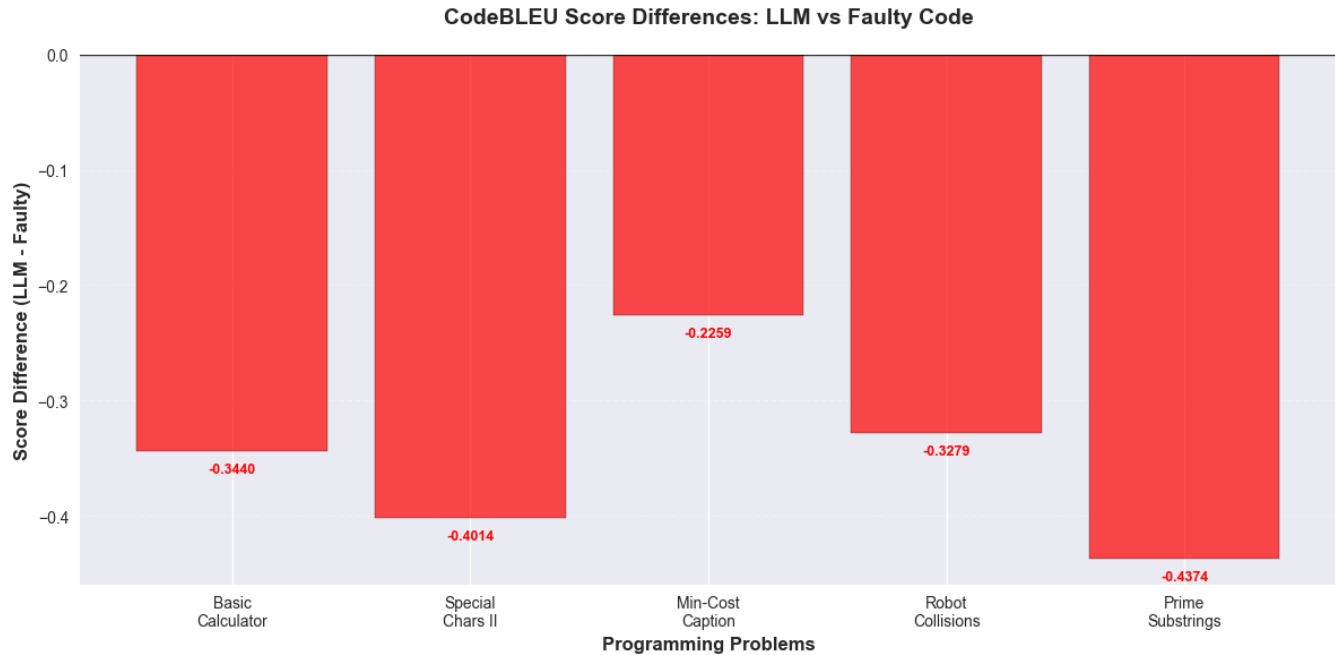


Key Findings:

- Average Correct vs LLM similarity: **0.3284**
- Average Correct vs Faulty similarity: **0.6757**
- Faulty code consistently shows higher similarity to correct code than LLM-generated code across all problems

3.2 Analysis of Unexpected Results

The results reveal a counter-intuitive pattern: faulty code receives higher CodeBLEU scores than LLM-generated code in all cases. This suggests that CodeBLEU may prioritize structural similarity over functional correctness.



For the "sum-of-largest-prime-substrings" problem, the difference is most pronounced (-0.4374), indicating that LLM-generated code implemented a substantially different approach while faulty code maintained similar structure with logical errors. The "minimum-cost-good-caption" problem shows the smallest difference (-0.2259), suggesting the LLM implementation was structurally closer to the correct solution.

4 Discussion: CodeBLEU and Semantic Consistency

4.1 Strengths of CodeBLEU

CodeBLEU demonstrates advantages over traditional metrics by capturing code-specific features:

- **AST matching** effectively identifies syntactic similarities in control structures
- **Data flow analysis** captures variable usage patterns
- **Weighted n-grams** emphasize important code elements

The metric shows high correlation with human evaluation in code synthesis tasks, with reported Pearson correlation coefficients up to 0.82 compared to 0.61 for standard BLEU.

4.2 Limitations in Semantic Evaluation

However, our results highlight important limitations [2]:

Structural Bias: CodeBLEU appears overly sensitive to surface-level similarities. Faulty code that maintains similar variable names, control structures, and code organization with minor logical

errors can achieve high scores, while functionally correct but structurally different implementations receive penalized scores [3].

Limited Semantic Understanding: The metric cannot detect runtime logic errors or functional equivalence between different algorithmic approaches. As noted in research, "BLEU considers n-gram precision, simply repeating the source text leads to a high score in monolingual tasks".

Implementation Variant Penalization: CodeBLEU may underestimate semantic consistency when correct functionality is achieved through different implementation approaches, such as alternative algorithms or coding styles.

4.3 Comparison with Execution-Based Metrics

Research indicates that static similarity metrics like CodeBLEU should be complemented with execution-based evaluation. Metrics such as:

- **Compilation success rate**
- **Unit test pass rates**
- **Functional correctness testing**

These dynamic metrics provide a more accurate assessment of semantic consistency by actually executing the code and verifying its behavior [4].

5 Conclusion and Recommendations

5.1 Conclusion

CodeBLEU provides valuable insights into code structural similarity but has limitations in accurately reflecting semantic consistency. Our experimental results demonstrate that CodeBLEU can assign higher scores to faulty code than to functionally correct LLM-generated code, indicating that the metric should not be used as the sole indicator of code quality. The metric effectively captures surface-level and syntactic similarities but may fail to distinguish between structurally similar but functionally different code implementations. This aligns with research findings that "metrics originate from the machine translation domain and it is unclear whether they are applicable for the code generation tasks".

5.2 Recommendations

For comprehensive code quality assessment, we recommend:

- **Multi-Metric Approach:** Combine CodeBLEU with execution-based metrics like test case pass rates to balance structural and functional evaluation.

- Human Evaluation: Include manual code review for critical applications, especially when CodeBLEU scores show unexpected patterns.
- Context-Aware Interpretation: Consider the specific programming problem and acceptable implementation variants when interpreting CodeBLEU scores.
- Threshold-Based Assessment: Establish problem-specific score thresholds rather than relying on absolute values, as acceptable similarity ranges may vary across different types of programming problems.

References

- [1] Ren, S., Guo, D., Lu, S., Zhou, L., Liu, S., Tang, D., Sundaresan, N., Zhou, M., Blanco, A., & Ma, S. (2020). CodeBLEU: a Method for Automatic Evaluation of Code Synthesis. *arXiv preprint arXiv:2009.10297*. This foundational paper introduces the CodeBLEU metric and explains its four-component framework combining n-gram matching, AST analysis, data flow analysis, and weighted keywords
- [2] Huang, J., et al. (2022). Execution-based Evaluation for Data Science Code Generation Models. *arXiv preprint arXiv:2211.09374*.
- [3] Mahmud, T., Duan, B., Pasareanu, C., & Yang, G. (2025). Enhancing LLM Code Generation with Ensembles: A Similarity-Based Selection Approach. *arXiv preprint arXiv:2503.15838*.
- [4] Chen, L., Guo, Q., Jia, H., Zeng, Z., Wang, X., Xu, Y., Wu, J., Wang, Y., Gao, Q., Wang, J., Ye, W., & Zhang, S. (2024). A Survey on Evaluating Large Language Models in Code Generation Tasks. *Journal of Computer Science and Technology*. arXiv preprint arXiv:2408.16498v1